MPOO 2027-1 · GRUPO 06

Parcial 1 · correcciones

Versión C del examen del 7 de octubre. Para cada pregunta vienen el enunciado, tu respuesta tal como la enviaste en el portal y su corrección. La calificación no va aquí: la ves con el profesor.

Antes de empezar. Este archivo lo subió el profesor a tu rama. Trae el cambio a tu computadora antes de tu siguiente push, o tu push va a fallar:

```
git checkout entregas_mendez_juan
git pull --no-rebase
```

Qué tienes que hacer. En hojas, **a mano**, copia cada pregunta y escribe debajo tu respuesta corregida, usando la corrección que viene aquí. Donde la corrección ya te da la salida o el código, no basta con copiarlo: escribe **por qué** sale así, con la tabla de seguimiento de las variables, la causa del error o lo que devuelve tu método en cada renglón de la tabla. Debajo de esas preguntas viene qué escribir. Escanea las hojas o tómales foto, júntalas en un PDF y súbelo a esta misma carpeta, `entregas/mendez_juan/examen/p1/`, con tu push de siempre.

Pregunta 0.1 · Preguntas abiertas

Pregunta

Contesta con tus palabras, en una a tres oraciones. Puedes apoyarte en un ejemplo pequeño de código
¿Qué es la abstracción al modelar un objeto? Da un ejemplo de un dato que conservarías y otro que omitirías según el contexto.

Tu respuesta

```
Es una herramienta que ayuda a hacer nuestro codigo mas simplificado y que ejecuta lo mismo
```

Corrección. La respuesta describe algo parecido a simplificar o refactorizar código, no la abstracción, y no da ejemplos. Abstraer es quedarse con lo esencial para el problema; por ejemplo, para calcular el área de un círculo se conserva el radio y se omite el color.

Pregunta 0.2 · Preguntas abiertas

Pregunta

¿Para qué sirve el constructor y en qué se distingue de un método común en su declaración?

Tu respuesta

```
El constructor es donde hacemos/creamos nuestro objeto basicamente , ejemplo de un objeto
llamado pizza se hace la clase,  primero van sus atributos, depues va el constructor que seria
tipo asi, public pizza{ string tamaño, string sabor}
this.tamaño
this.sabor
```

Corrección. La idea de que el constructor sirve para crear el objeto y asignar sus atributos con this es válida. Falta la distinción en la declaración: el constructor lleva el mismo nombre que la clase y no declara tipo de retorno, ni siquiera void.

Pregunta 0.3 · Preguntas abiertas

Pregunta

¿Qué impide final en una variable? Si guarda una referencia a un arreglo, ¿impide cambiar sus elementos?

Tu respuesta

```
depende de como se declare la variable
```

Corrección. 'Depende de cómo se declare' no responde lo que se pregunta. final impide volver a asignar la variable después de inicializarla; si guarda un arreglo, no se puede cambiar la referencia, pero sí sus elementos.

Pregunta 0.4 · Preguntas abiertas

Pregunta

¿Por qué == no basta para comparar el contenido de dos String? Explica qué usarías para comparar sus textos.

Tu respuesta

```
Creo que == no funciona en java para el contenido de string ya que es texto , yo usaria un
if/else o un booleano de true or false
```

Corrección. No explica por qué == falla, y if/else o un booleano no sirven para comparar textos. Con referencias, == compara si es el mismo objeto (identidad); para comparar el contenido se usa equals.

Pregunta 0.5 · Preguntas abiertas

Pregunta

¿En qué se distinguen while y do-while? Explica qué ocurre si la condición es falsa desde el inicio.

Tu respuesta

```
mientras y hacer mientras, si es falso no se ejecuta y se hace otra cosa
```

Corrección. Solo traduce los nombres. 'Si es falso no se ejecuta' es una idea válida para while, pero no distingue el caso de do-while. while revisa la condición antes y puede dar cero vueltas; do-while la revisa después y da al menos una vuelta aunque la condición sea falsa desde el inicio.

R1 · ¿Qué imprime?

Pregunta

Escribe la salida exacta, una línea por renglón. El fragmento está en un main válido.

```
int[][] tabla = {{9, 4, 7}, {2, 6, 8}};
int[] fila = tabla[0];
```

```
int total = 0;
for (int i = 0; i < fila.length; i++) {
    fila[i] = fila[i] / 2;
    total += tabla[0][i];
    System.out.println(i + ":" + total);
}
System.out.println((double) (total / 2));
System.out.println("P" + fila[0] + tabla[1][0]);
```

Tu respuesta

```
imprime el total dividido entre dos
y ademas
P la fila y tabla
```

Corrección. Escribe una descripción en vez de la salida exacta. Reconoces que la cuarta línea es total entre dos y la 'P' concatenada con fila y tabla en la quinta. La salida correcta es 0:4 / 1:6 / 2:9 / 4.0 / P42, porque fila es el mismo arreglo que tabla[0].

Qué escribir a mano. La salida ya la tienes en la corrección. Escribe por qué sale así: una tabla de seguimiento con el valor de cada variable en cada vuelta y en cada println.

R2 a) · ¿Compila, truena o funciona?

Pregunta

Incisos independientes: a/b en main; c en una clase válida. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
int[] datos = {3, 6};
int total = 0;
for (int i = 0; i <= datos.length; i++)
    total += datos[i];
System.out.println(total);
```

Tu respuesta

Clasificación

```
Funciona
```

Causa o salida

```
muestra los datos de total
```

Corrección. La clasificación es incorrecta: compila pero truena. Con i <= datos.length el ciclo intenta leer datos[2], fuera del arreglo (ArrayIndexOutOfBoundsException), y el println nunca se ejecuta.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R2 b) · ¿Compila, truena o funciona?

Pregunta

Incisos independientes: a/b en main; c en una clase válida. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
Integer piezas = 6;
System.out.println(piezas + 2);
System.out.println("P" + piezas);
```

Tu respuesta

Clasificación

```
No compila
```

Causa o salida

```
integer? no existe
```

> **Corrección.** Integer sí existe: es la clase envolvente de int, y el código funciona. Imprime 8 / P6: en piezas + 2 se desempaqueta para sumar, y con texto se concatena.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R2 c) · ¿Compila, truena o funciona?

Pregunta

Incisos independientes: a/b en main; c en una clase válida. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
static int medir(int valor) {
    if (valor > 0) return valor;
}
```

Tu respuesta

Clasificación

```
Funciona
```

Causa o salida

```
si valor es menor que 0 retorna
```

> **Corrección.** La clasificación es incorrecta: no compila. Cuando valor es cero o negativo no hay return, y Java exige que todos los caminos de un método int devuelvan un valor.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R3 · Modelado: base, reto y justificación

Pregunta

Cada lámpara tiene etiqueta fija y nace apagada. Cuenta las lámparas creadas. Escribe encender() y una consulta del estado; cada lámpara cambia por separado. Usa el enum dado.

```
enum Estado { APAGADO, ENCENDIDO, PARPADEO }
```

Base. Declara al menos dos atributos de instancia con tipo, uno static y una constante final (¿también static?). Completa constructor y dos métodos: cambio y consulta.

Reto. ¿Qué tipo tiene el atributo de estado y con qué valor nace?

Justifica. Una línea por decisión: tipo, instancia/clase, final o mutable, constructor y métodos.

Cuerpos de 1 a 3 líneas, sin estructuras de control ni main. Hay más de un modelo correcto. Un elemento sin justificar vale la mitad.

Tu respuesta

Clase

```
public class lamparas {
//atributos
int numlamparas
string estado
string encender

//constructor
public lamparas { int numlamparas, string estado, string encender}
this.numlamparas
this.estado


//metodos
public string encender (){
if (encender = si ) { "encendido"
}
else { "apagado"
}







public static void







public main { static arg[]}
system.out.println("Lampara:",numlampara, "Estado:", estado)
```

Justificaciones

```
creo mis atributos que son el numero de lamparas, el estado y el encender,
hago el constructor osea creo mi objeto y al final los metodos, que hara encender si en si la
```

```
lampara estara encendida si es otro no lo estara
```

Corrección. Declara atributos de instancia con tipo, pero el estado es String y no el enum Estado, falta la etiqueta final y el contador no es static. El constructor está incompleto (this.numlamparas sin asignar) y no fija APAGADO ni incrementa el contador. encender usa if y no asigna Estado.ENCENDIDO, y falta el método de consulta. No hay constante final y el reto quedó sin responder: estado es de tipo Estado y nace en Estado.APAGADO.

R4 · Código con repetición

Pregunta

Devuelve la posición de la menor temperatura usando for; si se repite, devuelve la primera. Los índices empiezan en 0. El arreglo no es null, tiene al menos un elemento y puede incluir negativos. No lo modifiques.

```
static int posicionMinima(int[] temperaturas)
```

Ejemplos (argumentos → retorno):

{8, 3, 5} → 1 · {7} → 0 · {-2, -8, -1} → 1

Escribe el método completo. No imprimas ni leas del teclado.

Tu respuesta

```
static int posicionMinima(int[] temperaturas)
int[] temperaturas = { 1, 2, 3}
int total= 0
for (int i = 0; i<= datos.lenght; i++}
total += datos[i];
```

Corrección. La cabecera es correcta y usa for. El cuerpo vuelve a declarar el parámetro, suma un arreglo 'datos' que no existe con i <= length y no busca el mínimo ni devuelve nada. Lo que sí está bien es que no imprime. Lo correcto era tomar posicion = 0, recorrer desde i = 1 y, si temperaturas[i] < temperaturas[posicion], hacer posicion = i; al final, return posicion.

Qué escribir a mano. El método corregido completo y, para cada ejemplo o renglón de la tabla del enunciado, qué devuelve tu método y por qué.

R5 · Código con selección

Pregunta

Un equipo decide su mantenimiento según los ciclos de uso. Un valor menor o igual que 0 es "INVALIDO". Para los positivos, aplica la primera regla que corresponda, en el orden de la tabla. Usa if / else-if / else. No imprimas ni leas del teclado.

```
static String mantenimiento(int ciclos)
```

Regla o valor recibido	Texto devuelto
Menor o igual que 0	"INVALIDO"
Divisible entre 24	"COMPLETO"
Si no, divisible entre 12	"REVISION"
Si no, divisible entre 4	"BASICO"

Cualquier otro	"NINGUNO"

Tu respuesta

```
if ( ciclos <= 0 )
{      "invalido"
}
else { "valido"
}

if ( ciclos/24 )
{     "Completo"
}
else{ "incompleto"
}

if (ciclos /12 )
{ "revision"
}
else "no" }

if ( ciclos/4)
{ "Basico"
}
else "ninguno"
}
```

Corrección. Falta la cabecera y usa if/else separados en lugar de una cadena if / else-if / else. Valida bien ciclos <= 0 como inválido y respeta el orden 24, 12, 4. Para la divisibilidad usa ciclos/24 en vez de ciclos % 24 == 0. No hay ningún return, los textos no coinciden con los pedidos ("COMPLETO", "REVISION", etc.) y las llaves no cierran.

Qué escribir a mano. El método corregido completo y, para cada ejemplo o renglón de la tabla del enunciado, qué devuelve tu método y por qué.